Visual Odometry monoculară cu ORB features

Keypoint detection și matching pentru estimarea poziției unei drone

Lab 2 — note de curs pentru `12_visual_odometry_orb.py`

Robotics Specialization — Aerial Robotics

Rezumat

Aceste note explică modul în care o dronă își poate estima propria mișcare din imaginile unei singure camere de la bord. Construim un pipeline de *visual odometry* (VO) complet, dar intenționat simplu. Accentul cade pe primii doi pași, de care depind toți ceilalți: **keypoint detection** cu ORB și **matching**-ul descriptorilor între imagini. Pașii geometrici care urmează (essential matrix cu RANSAC și recoverPose) sunt prezentați pe scurt. Metoda este aplicată pe un zbor real al unei drone Crazyflie cu cameră AI-deck (324×244 pixeli, grayscale, aproximativ 8,3 cadre pe secundă) și validată cu un sistem de motion capture OptiTrack.

Cuprins

1 Introducere **3**
- 1.1 Ce este visual odometry? . . . 3
- 1.2 Obiective . . . 3
- 1.3 Pipeline-ul pe scurt . . . 3
- 1.4 Datele . . . 3

2 Noțiuni de bază: camera **4**

3 Keypoint detection **4**
- 3.1 Ce este un keypoint bun? . . . 4
- 3.2 ORB pe scurt . . . 4
- 3.3 Detecție: FAST . . . 5
- 3.4 Ordonarea candidaților: Harris corner measure . . . 5
- 3.5 Invarianță la scară: image pyramid . . . 5
- 3.6 Invarianță la rotație: orientarea keypoint-ului . . . 6
- 3.7 Descriptorul: rotated BRIEF . . . 6
- 3.8 Pre-procesare: CLAHE și self mask . . . 7

4 Matching **8**
- 4.1 Compararea descriptorilor: Hamming distance . . . 8
- 4.2 Brute force nearest neighbours . . . 8
- 4.3 Lowe's ratio test . . . 9
- 4.4 Rezultate pe date reale . . . 9
- 4.5 Alegerea pragului ρ . . . 9

5 Pasul următor: essential matrix cu RANSAC **10**

6 Pasul următor: R, t/‖t‖ cu recoverPose **11**

7 De la pași la traiectorie **12**

8 Rezultate pe un zbor real **13**

9 Limitări și ce fac sistemele reale **13**

10 Exerciții: ablation study **14**

A Rularea codului **16**

1 Introducere

1.1 Ce este visual odometry?

Odometry înseamnă estimarea modului în care se schimbă pose-ul unui robot în timp, folosind senzori proprii. *Wheel odometry* numără rotațiile roților; *inertial odometry* integrează accelerațiile și vitezele unghiulare. *Visual odometry* estimează mișcarea unei camere din felul în care se schimbă imaginea unei scene statice de la un cadru la altul. Termenul a fost popularizat de Nistér et al. în 2004. Astăzi, VO este o componentă de bază pentru drone, rovere planetare, căști AR și mașini autonome.

Pentru o dronă mică, în interior, VO este atractivă: nu există GPS în interior, o cameră cântărește doar câteva grame și este deja la bord pentru alte sarcini (urmărirea unei persoane, detecția obstacolelor).

1.2 Obiective

După parcurgerea acestor note și a codului ar trebui să puteți:

1. explica ce face un keypoint bun și de ce colțurile sunt candidați buni;
2. descrie pașii ORB: FAST, Harris corner measure, image pyramid, orientarea și descriptorul BRIEF;
3. explica cum se compară descriptorii binari cu Hamming distance;
4. explica de ce cel mai apropiat vecin nu este suficient și cum ajută Lowe's ratio test;
5. descrie pe scurt cum se obțin, din match-uri, rotația și direcția translației camerei;
6. explica de ce o singură cameră nu poate măsura scara;
7. evalua experimental, printr-un ablation study, cât contează fiecare componentă a pipeline-ului.

1.3 Pipeline-ul pe scurt

Figura 1 arată algoritmul complet. Casetele evidențiate sunt subiectul principal al acestor note.

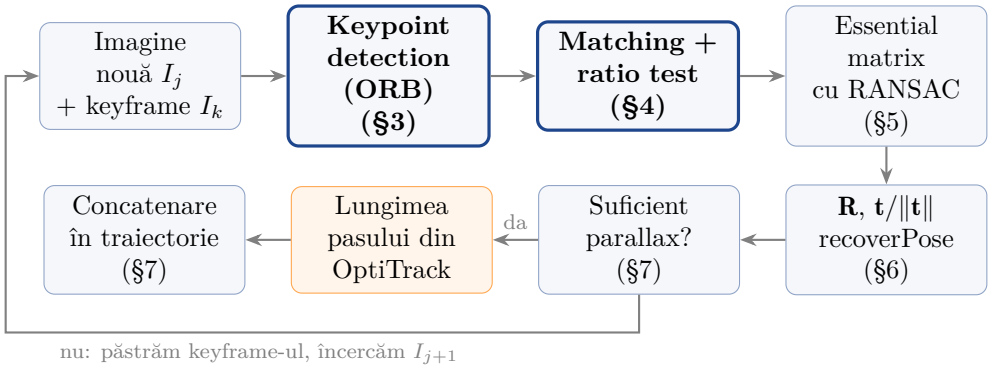


Figura 1: Pipeline-ul de visual odometry. Casetele albastre folosesc doar camera; caseta portocalie este singurul loc în care intră ground truth-ul. Casetele îngroșate sunt subiectul principal.

1.4 Datele

Înregistrarea `stream_out/imgs/1794489705263` conține 829 de imagini JPEG (100 s) transmise de AI-deck și un fișier `positions.csv` care stochează, pentru fiecare imagine, momentul de timp și poziția OptiTrack (x,y,z) și yaw-ul dronei. Fișierul `ranging/camera_calibration.json` conține distanța focală $f = 177{,}6$ px, un coeficient de distorsiune radială $k_1 = -0{,}135$, o înclinare a camerei în jos de 6° și o latență video de 85 ms față de OptiTrack. Imaginea binară `ranging/self_mask.png` marchează pixelii în care se văd rama și elicele dronei.

2 Noțiuni de bază: camera

Un punct $\mathbf{X} = (X,Y,Z)^\top$ exprimat în sistemul de coordonate al camerei (x la dreapta, y în jos, z de-a lungul axei optice) se proiectează în pixelul (u,v) al imaginii:

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{KX}, \qquad \mathbf{K} = \begin{pmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{pmatrix}. \tag{1}$$

$\mathbf{K}$ conține *intrinsics*: distanța focală f în pixeli și centrul imaginii (c_x,c_y). Adâncimea Z se pierde în proiecție; de aici vine problema scării din Secțiunea 6.

Eliminând $\mathbf{K}$ și distorsiunea lentilei obținem **normalised coordinates** $\mathbf{x} = (X/Z, Y/Z, 1)^\top$, adică coordonatele unei camere ideale cu $f = 1$. Toată geometria din Secțiunile 5 și 6 folosește aceste coordonate. Keypoint detection și matching lucrează însă direct pe pixeli.

În cod

`cv2.undistortPoints(pts, K, dist)` transformă coordonatele în pixeli în normalised coordinates.

3 Keypoint detection

3.1 Ce este un keypoint bun?

Pentru a măsura mișcarea avem nevoie de puncte pe care le putem **regăsi** în imaginea următoare. Candidații buni sunt **colțurile (corners)**: patch-uri mici al căror aspect se schimbă în orice direcție le-am deplasa. Într-o zonă uniformă, o deplasare nu schimbă nimic; pe o muchie dreaptă, o deplasare de-a lungul muchiei nu schimbă nimic. Ambele sunt ambigue: aceasta este **aperture problem** (Figura 2).

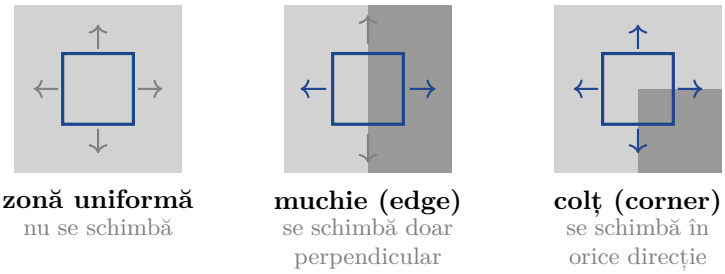


Figura 2: Deplasăm fereastra (albastru) în cele patru direcții. Săgețile albastre marchează direcțiile în care conținutul ferestrei se schimbă. Doar colțul poate fi localizat în ambele direcții.

3.2 ORB pe scurt

ORB (*Oriented FAST and Rotated BRIEF*, Rublee et al. 2011) este o alternativă rapidă și gratuită la SIFT și SURF. Are două părți, care răspund la două întrebări diferite:

Detector: oriented FAST *Unde* se află keypoints?	**Descriptor: rotated BRIEF** *Cum arată* fiecare keypoint?
FAST găsește colțuri candidate (§3.3)	un șir binar de 256 de biți (32 bytes)
Harris le ordonează după calitate (§3.4)	fiecare bit este o comparație de intensitate
image pyramid oferă invarianță la scară (§3.5)	este rotit cu orientarea keypoint-ului
intensity centroid dă orientarea (§3.6)	(§3.7)

Ambele părți folosesc doar operații simple, deci ORB rulează în timp real chiar și pe un calculator modest.

3.3 Detecție: FAST

FAST (Rosten și Drummond, 2006) examinează cei **16 pixeli** de pe un cerc de rază 3 în jurul unui pixel candidat p (Figura 3). Pixelul p este un colț dacă cel puțin **9 pixeli consecutivi** de pe cerc sunt toți mai luminoși decât $I(p) + \tau$, sau toți mai întunecați decât $I(p) - \tau$. Folosim $\tau = 10$ niveluri de gri (`fastThreshold`).

FAST folosește doar comparații de intensitate, fără derivate și fără înmulțiri; de aceea este foarte rapid. Pragul τ controlează sensibilitatea: un τ mic găsește mai multe colțuri, dar și mai multe colțuri „false", produse de zgomot.

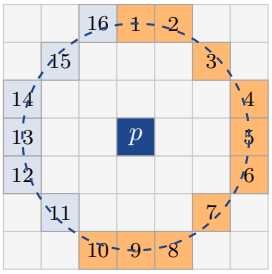


Figura 3: Cercul FAST de 16 pixeli în jurul candidatului p. Pixelii 1–10 (portocaliu) sunt mai luminoși decât $I(p) + \tau$: sunt 10 pixeli consecutivi, cel puțin 9, deci p este un colț.

3.4 Ordonarea candidaților: Harris corner measure

FAST răspunde puternic și de-a lungul muchiilor. De aceea candidații sunt ordonați după **Harris corner measure**, calculat din **structure tensor M** al unei ferestre $\mathcal{W}$ din jurul pixelului:

$$\mathbf{M} = \sum_{(x,y)\in\mathcal{W}} \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}, \qquad R = \det(\mathbf{M}) - k\,\mathrm{tr}(\mathbf{M})^2 = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2, \tag{2}$$

unde I_x, I_y sunt derivatele imaginii, λ_1,λ_2 valorile proprii ale lui $\mathbf{M}$, iar $k \approx 0{,}04$ este o constantă empirică. Valorile proprii măsoară cât de mult variază intensitatea în cele două direcții principale ale ferestrei:

Valorile proprii λ_1, λ_2	Tip de regiune	R
ambele mici	zonă uniformă	≈ 0
una mare, una mică	muchie (edge)	< 0
ambele mari	colț (corner)	> 0, mare

Păstrăm doar cei mai buni $N = 1000$ de candidați (`nfeatures`).

3.5 Invarianță la scară: image pyramid

Același obiect apare mai mare sau mai mic când drona se apropie sau se depărtează. Un colț detectat într-un patch de câțiva pixeli poate deveni, la o altă distanță, o curbă netedă. De aceea detecția se repetă pe un **image pyramid**: 8 niveluri, fiecare de 1,2 ori mai mic decât precedentul (Figura 4). Un keypoint de pe un nivel grosier acoperă o zonă mare din imaginea originală. Între primul și ultimul nivel, diferența de scară este $1{,}2^7 \approx 3{,}6$.

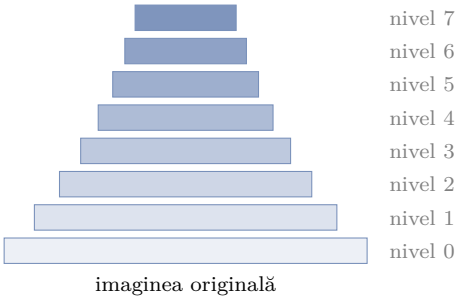


Figura 4: Image pyramid cu 8 niveluri; fiecare nivel este de 1,2 ori mai mic decât cel de sub el.

3.6 Invarianță la rotație: orientarea keypoint-ului

Fiecare keypoint primește o **orientare**, din **intensity centroid** al patch-ului său (Figura 5). Cu momentele

$$m_{pq} = \sum_{x,y} x^p\, y^q\, I(x,y), \qquad \theta = \operatorname{atan2}(m_{01}, m_{10}), \tag{3}$$

unde x, y sunt măsurate față de centrul patch-ului. Vectorul (m_{10}, m_{01}) arată direcția în care patch-ul este „mai luminos", iar θ este unghiul acestei direcții. Dacă imaginea se rotește, θ se rotește la fel. Descriptorul se calculează apoi într-un sistem de referință rotit cu θ, deci nu depinde de rotația în plan a imaginii.

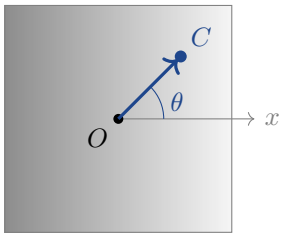


Figura 5: Patch-ul este mai luminos spre dreapta-sus. O: centrul keypoint-ului; C: intensity centroid. Orientarea θ este unghiul vectorului OC.

3.7 Descriptorul: rotated BRIEF

Detectorul spune *unde* se află un keypoint. Pentru a-l regăsi în altă imagine avem nevoie și de o descriere a zonei din jurul lui: **descriptorul**. ORB folosește o variantă a lui **BRIEF** (Calonder et al. 2010), care transformă patch-ul într-un șir de **256 de biți**.

Pașii concreți.

1. Luăm un patch de 31×31 pixeli centrat în keypoint.
2. Netezim patch-ul. Imaginile sunt zgomotoase, iar un singur pixel poate „sări" cu câteva niveluri de gri de la un cadru la altul; după netezire, fiecare pixel reprezintă media unei mici zone, iar comparațiile devin stabile.
3. Comparăm intensitățile a 256 de perechi fixe de pixeli $(\mathbf{a}_i, \mathbf{b}_i)$ (Figura 6). Lista de perechi este *aceeași* pentru toți keypoints, în toate imaginile:

$$\text{bit}_i = \begin{cases} 1, & I(\mathbf{a}_i) < I(\mathbf{b}_i) \\ 0, & \text{altfel} \end{cases} \qquad i = 1, \ldots, 256. \tag{4}$$

4. Cei 256 de biți (32 bytes) formează descriptorul.

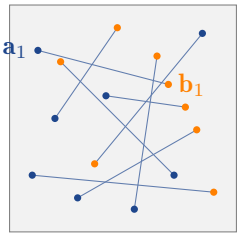


Figura 6: Opt dintre cele 256 de perechi de pixeli dintr-un patch de 31×31. Fiecare pereche dă un bit.

Exemplu

Să presupunem că patch-ul are, după netezire, următoarele intensități la primele patru perechi:

Pereche	$I(\mathbf{a})$	$I(\mathbf{b})$	$I(\mathbf{a}) < I(\mathbf{b})$?	Bit
1	80	120	da	1
2	150	90	nu	0
3	60	200	da	1
4	110	105	nu	0

Descriptorul începe cu `1 0 1 0` …. În cadrul următor, același colț poate fi puțin mai luminos, de exemplu cu 20 de niveluri peste tot. Comparațiile *nu se schimbă* ($100 < 140$ rămâne adevărat), deci descriptorul rămâne aproape identic. BRIEF reține ordinea intensităților, nu valorile lor, și de aceea tolerează bine schimbările de iluminare.

Ce adaugă ORB peste BRIEF-ul original. BRIEF-ul simplu are o problemă: dacă imaginea se rotește, perechile fixe cad pe alți pixeli și descriptorul se schimbă complet. ORB rezolvă asta în două moduri:

- **rotește perechile** $(\mathbf{a}_i, \mathbf{b}_i)$ cu orientarea θ a keypoint-ului (Secțiunea 3.6), astfel încât ele sunt mereu așezate la fel față de conținutul patch-ului;
- **alege perechile prin învățare**: au fost păstrate perechile care dau biți *necorelați* între ei și *discriminativi*, adică biți care deosebesc bine keypoints diferite. Altfel, mulți biți ar repeta aceeași informație.

3.8 Pre-procesare: CLAHE și self mask

Imaginile AI-deck sunt mici, cu contrast redus și zgomot. Două măsuri simple ajută (Figura 7):

- **CLAHE** (*contrast limited adaptive histogram equalisation*) mărește contrastul local, deci FAST găsește mai multe colțuri;
- **self mask** exclude rama și elicele dronei. Acești pixeli se mișcă *odată cu* camera: matchurile lor au deplasare zero și ar trage estimarea spre „nicio mișcare”.

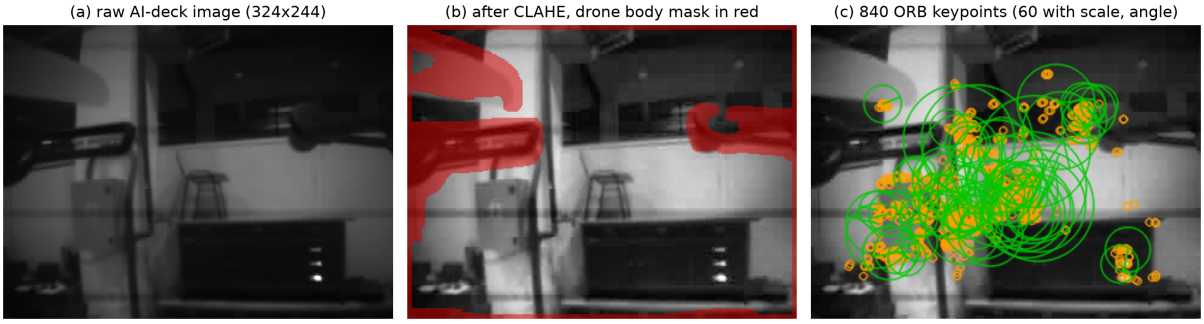


Figura 7: Keypoint detection pe cadrul 282. (a) Imaginea brută. (b) După CLAHE; zona roșie este self mask, unde nu se detectează keypoints. (c) ORB keypoints (portocaliu); 60 dintre ei, aleși aleator, sunt desenați cu scara (cercul) și orientarea (raza). Cercurile mari provin din nivelurile grosiere ale piramidei.

În cod

```
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(4, 4))
orb = cv2.ORB_create(nfeatures=1000, fastThreshold=10)

img = clahe.apply(gray)                    # mai mult contrast = mai multe keypoints
kp, des = orb.detectAndCompute(img, mask)  # mask = 0 pe corpul dronei
```

`kp` este lista de keypoints, fiecare cu poziție, scară (nivelul din piramidă) și orientare. `des` este matricea descriptorilor, $N \times 32$ bytes, câte un rând pentru fiecare keypoint.

4 Matching

Acum avem, în fiecare imagine, o listă de keypoints cu descriptorii lor. Matching-ul caută, pentru fiecare keypoint din keyframe, keypoint-ul din imaginea nouă care corespunde aceluiași punct din scenă.

4.1 Compararea descriptorilor: Hamming distance

Doi descriptori binari se compară cu **Hamming distance**, numărul de biți care diferă:

$$d(\mathbf{a},\mathbf{b}) = \text{popcount}(\mathbf{a} \oplus \mathbf{b}) \in \{0,\ldots,256\}. \tag{5}$$

Exemplu

```
a       = 1 0 1 1 0 0 1 0
b       = 1 0 0 1 1 0 1 1
a XOR b = 0 0 1 0 1 0 0 1    ->   d(a, b) = 3
```

Calculatorul evaluează această distanță cu o singură operație XOR și o numărare de biți (popcount), mult mai ieftin decât Euclidean distance dintre descriptorii float ai SIFT. Pentru același punct din scenă, puțini biți diferă (de exemplu $d \approx 20$). Pentru puncte diferite, biții coincid doar întâmplător, deci $d \approx 128$, jumătate din biți.

4.2 Brute force nearest neighbours

Pentru fiecare descriptor din keyframe căutăm cei **doi cei mai apropiați** descriptori din imaginea nouă, cu distanțele $d_1 \leq d_2$ ($k = 2$). Doar cel mai apropiat vecin nu este suficient, pentru că

produce multe match-uri greșite:

- **structuri repetate**: sertarele dulapului sau ramele ferestrelor arată la fel;
- **puncte fără pereche**: keypoint-ul a ieșit din câmpul vizual sau este acoperit. Cel mai apropiat vecin există întotdeauna, chiar dacă perechea corectă lipsește.

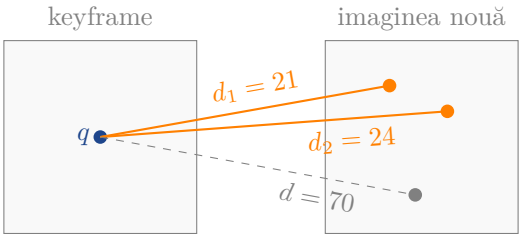


Figura 8: Pentru descriptorul q există doi candidați aproape la fel de buni: match-ul este ambiguu.

4.3 Lowe's ratio test

Un match este acceptat doar dacă este **clar mai bun** decât al doilea candidat (Lowe, 2004):

$$d_1 < \rho\, d_2, \qquad \rho = 0{,}75. \tag{6}$$

Exemplu	d_1	d_2	d_1/d_2	Decizie
clar	20	60	0,33	acceptat
ambiguu	21	24	0,88	respins

Dacă cei doi candidați sunt aproape la fel de buni, match-ul este ambiguu și îl eliminăm. Testul elimină și multe puncte fără pereche: pentru ele, toți candidații sunt la fel de depărtați, deci $d_1 \approx d_2$. Testul a fost introdus inițial pentru descriptorii SIFT, dar funcționează la fel de bine pentru descriptorii binari.

4.4 Rezultate pe date reale

Figura 9 arată efectul pe o pereche reală de imagini: cele 866 de match-uri nearest neighbour se reduc la 283 după ratio test, iar verificarea geometrică din Secțiunea 5 păstrează 227 dintre ele (RANSAC inliers).

4.5 Alegerea pragului ρ

	Efect	Risc
ρ mic (ex. 0,6)	puține match-uri, dar fiabile	prea puține pentru o estimare stabilă
ρ mare (ex. 0,9)	multe match-uri	mai mulți outliers; RANSAC lucrează mai mult

Valoarea folosită, $\rho = 0{,}75$, este un compromis obișnuit între numărul de match-uri și fiabilitatea lor.

(a) nearest neighbour for every descriptor: 866 matches



(b) after the ratio test: 283 matches; RANSAC keeps 227 inliers (green), rejects 56 (red)

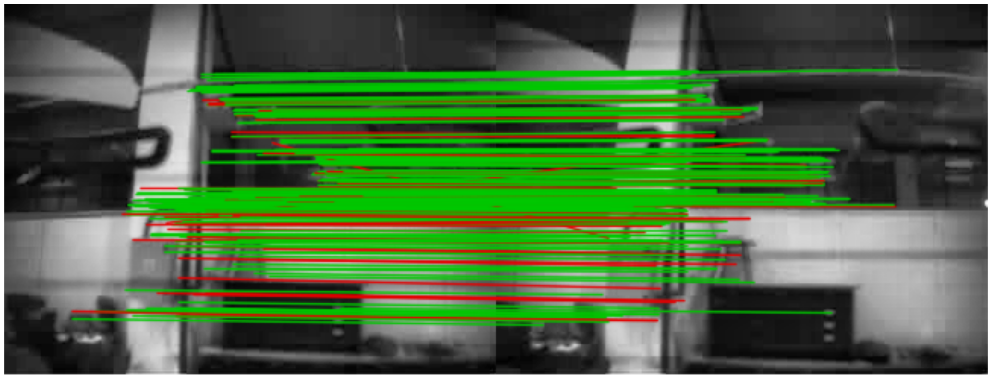


Figura 9: Matching între keyframe-ul 300 și cadrul 303. (a) Cel mai apropiat vecin pentru fiecare descriptor. (b) După ratio test; match-urile verzi sunt compatibile cu o singură mișcare rigidă a camerei (RANSAC inliers), cele roșii sunt respinse ca outliers.

În cod

```
matcher = cv2.BFMatcher(cv2.NORM_HAMMING)    # descriptori binari -> Hamming distance
pairs = matcher.knnMatch(des1, des2, k=2)    # cei 2 vecini cei mai apropiați

good = [m for m, n in pairs if m.distance < 0.75 * n.distance]   # ratio test
pts1 = [kp1[m.queryIdx].pt for m in good]
pts2 = [kp2[m.trainIdx].pt for m in good]
```

`queryIdx` și `trainIdx` sunt indicii keypoints în cele două imagini; `pts1` și `pts2` sunt perechile de pixeli care merg mai departe în pipeline. În script (funcția `match`), perechile cu mai puțin de 2 vecini sunt eliminate înainte de ratio test.

5 Pasul următor: essential matrix cu RANSAC

Un punct 3D static are coordonatele $\mathbf{X}_1$ în camera 1 și $\mathbf{X}_2$ în camera 2, legate de o mișcare rigidă $\mathbf{X}_2 = \mathbf{R}\,\mathbf{X}_1 + \mathbf{t}$. Cele două centre ale camerelor și punctul formează un triunghi, deci se află în același plan, **epipolar plane** (Figura 10). Din această coplanaritate rezultă **epipolar constraint**, valabil pentru fiecare match corect $(\mathbf{x}_1, \mathbf{x}_2)$ în normalised coordinates:

$$\boxed{\mathbf{x}_2^\top \mathbf{E}\,\mathbf{x}_1 = 0, \qquad \mathbf{E} = [\mathbf{t}]_\times \mathbf{R}} \tag{7}$$

unde $[\mathbf{t}]_\times$ este matricea produsului vectorial ($[\mathbf{t}]_\times \mathbf{v} = \mathbf{t} \times \mathbf{v}$). $\mathbf{E}$ este **essential matrix** (Longuet-Higgins, 1981). Ea conține rotația $\mathbf{R}$ (3 grade de libertate) și *direcția* translației $\mathbf{t}$ (2 grade de

libertate), deci sunt necesare minim 5 match-uri (five-point algorithm, Nistér 2004).

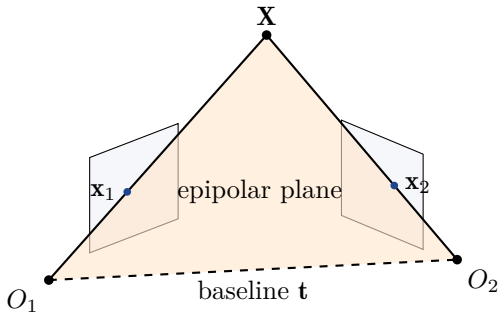


Figura 10: Geometria epipolară: cele două centre ale camerelor, punctul 3D și ambele puncte din imagine se află în același plan.

Chiar și după ratio test, unele match-uri sunt greșite, iar un singur outlier poate strica o estimare least squares. **RANSAC** (Fischler și Bolles, 1981) rezolvă problema astfel:

1. alegem 5 match-uri aleatoare și calculăm candidatul $\mathbf{E}$;
2. numărăm inliers: match-urile a căror distanță față de epipolar line este sub 1 pixel;
3. repetăm de N ori și păstrăm candidatul cu cei mai mulți inliers, apoi îl rafinăm pe toți inliers.

Un eșantion cu un outlier dă un $\mathbf{E}$ greșit, cu care puține alte match-uri sunt de acord; un eșantion curat dă $\mathbf{E}$-ul corect, cu care sunt de acord toate match-urile corecte. Pentru o probabilitate de succes de 99,9 % și jumătate din match-uri corecte, sunt suficiente aproximativ 218 iterații.

În cod

```
E, mask = cv2.findEssentialMat(n1, n2, np.eye(3), method=cv2.RANSAC,
                               prob=0.999, threshold=ransac_px / f)
```

Punctele sunt deja normalizate, deci matricea camerei este identitatea, iar pragul de 1 pixel este împărțit la f. `mask` marchează RANSAC inliers.

6 Pasul următor: R, t/‖t‖ cu recoverPose

Din $\mathbf{E}$ extragem mișcarea camerei în trei pași:

1. **SVD**: $\mathbf{E} = \mathbf{U}\,\mathrm{diag}(1,1,0)\,\mathbf{V}^\top$ dă două rotații posibile și $\mathbf{t} = \pm\mathbf{u}_3$ (ultima coloană a lui $\mathbf{U}$), deci 4 candidați;
2. **cheirality check**: triangulăm punctele și păstrăm candidatul care le plasează în fața ambelor camere (adâncime pozitivă);
3. **rezultat**: rotația $\mathbf{R}$ și direcția translației $\mathbf{t}/\|\mathbf{t}\|$, cu $\|\mathbf{t}\| = 1$.

Ecuația $\mathbf{X}_2 = \mathbf{R}\mathbf{X}_1 + \mathbf{t}$ descrie mișcarea *punctelor* față de cameră; camera se mișcă invers: $\mathbf{R}_C = \mathbf{R}^\top$, $\mathbf{t}_C = -\mathbf{R}^\top\mathbf{t}$. În script, aceste mărimi sunt apoi exprimate în sistemul de coordonate al dronei (body frame), ținând cont de înclinarea camerei.

Ideea cheie

Dacă înmulțim toate punctele și translația cu același factor $\alpha > 0$, imaginile nu se schimbă, deoarece proiecția (1) împarte la adâncime. O casă de două ori mai mare și de două ori mai departe arată la fel. O singură cameră măsoară rotația și *direcția* translației, dar nu *lungimea* ei: monocular VO este definită doar până la o scară necunoscută.

În cod

```
_, R, t, _ = cv2.recoverPose(E, n1, n2, np.eye(3), mask=mask)
```

Capcană

`recoverPose` returnează și o mască, dar aceasta păstrează doar punctele aflate în fața camerelor *și* mai aproape de 50 de baseline-uri. Pentru mișcările mici ale dronei, aproape fiecare punct este „departe”, așa că această mască arată rău chiar și când pasul este corect. Masca RANSAC din `findEssentialMat` este măsura corectă a calității.

7 De la pași la traiectorie

Scara. În practică, scara vine dintr-o sursă suplimentară: o cameră stereo, un IMU, un senzor de înălțime sau un obiect de dimensiune cunoscută. Pentru a ne concentra pe partea vizuală, luăm **lungimea fiecărui pas din OptiTrack**, $s_k = \|\mathbf{p}^{\mathrm{OT}}(t_j) - \mathbf{p}^{\mathrm{OT}}(t_k)\|$. Doar acest număr vine din OptiTrack; direcția fiecărui pas și toate rotațiile vin din cameră.

Keyframes. La 8,3 cadre pe secundă, drona se deplasează doar 1–5 cm între două cadre. Un punct aflat la 3 m se deplasează atunci în imagine cu doar $\Delta u \approx f\,b/Z = 177{,}6 \cdot 0{,}03/3 \approx 1{,}8$ px, cam cât precizia unui ORB keypoint pe aceste imagini zgomotoase. Direcția translației se pierde în zgomot (Figura 11). De aceea comparăm fiecare imagine nouă cu ultimul **keyframe**, nu cu imaginea anterioară, și acceptăm pasul doar când **parallax**-ul (deplasarea rămasă după eliminarea rotației) atinge 4 px sau când au trecut 8 cadre. Dacă sunt mai puțin de 20 de inliers, pasul este sărit.

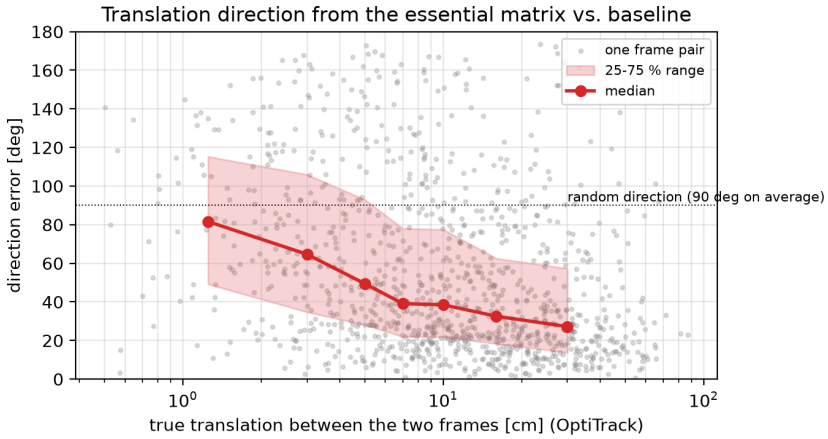


Figura 11: Unghiul dintre direcția translației estimate de VO și cea din OptiTrack, în funcție de baseline-ul real. Eroarea scade pe măsură ce baseline-ul crește.

Integrarea. Pose-ul dronei în lume este o rotație $\mathbf{R}_{WB}$ și o poziție $\mathbf{p}$. Fiecare pas relativ $(\mathbf{R}_B, s\,\hat{\mathbf{t}}_B)$ se compune cu pose-ul curent:

$$\mathbf{p}_j = \mathbf{p}_k + \mathbf{R}_{WB,k}\,(s_k\,\hat{\mathbf{t}}_B), \qquad \mathbf{R}_{WB,j} = \mathbf{R}_{WB,k}\,\mathbf{R}_B. \tag{8}$$

Erorile fiecărui pas se adună și nu sunt corectate niciodată: de aceea orice sistem VO are **drift**. O eroare de rotație este cea mai dăunătoare, deoarece rotește toți pașii următori.

8 Rezultate pe un zbor real

VO pornește de la poziția și yaw-ul OptiTrack la primul cadru în zbor și nu mai primește apoi nicio poziție sau orientare din OptiTrack. Tabelul 1 și Figura 12 rezumă zborul de 77 s și 22,5 m.

	cadru cu cadru	cu keyframes	rămâne la start
pași acceptați	636	169	—
RANSAC inliers (mediană)	147	85	—
eroare 3D mediană [m]	1,94	**1,04**	3,03
eroare 3D finală [m]	2,30	**1,70**	3,85

Tabela 1: Eroarea de poziție față de OptiTrack, fără nicio corecție după start. „Rămâne la start” este un estimator trivial care presupune că drona nu s-a mișcat deloc.

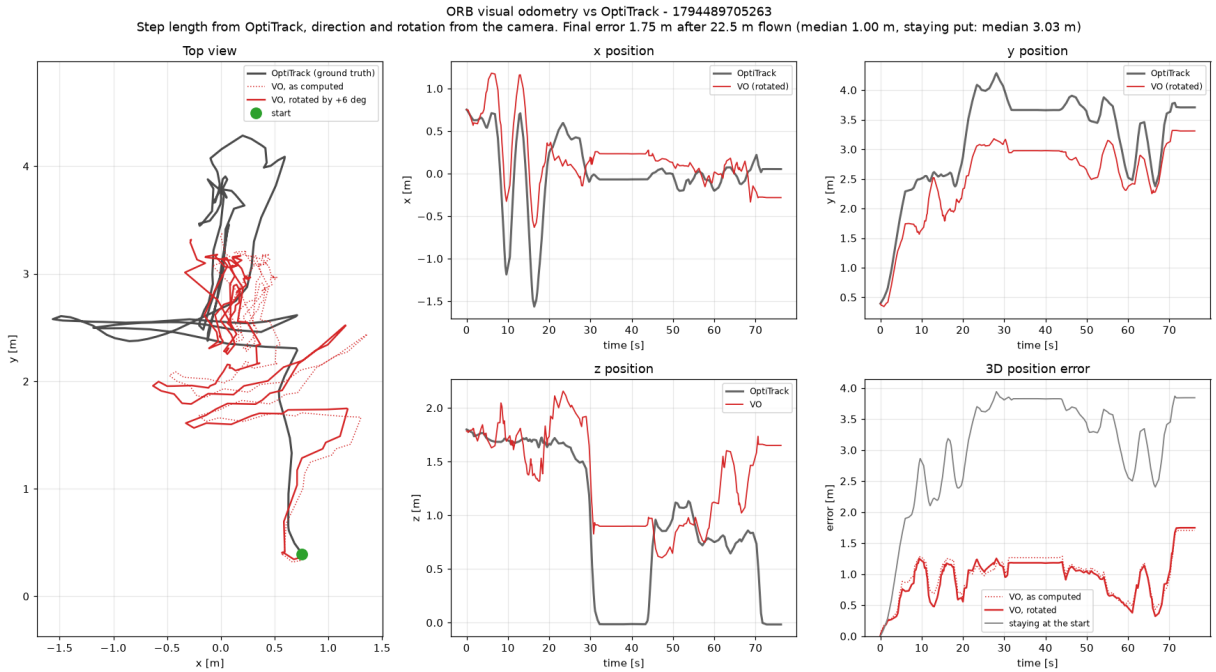


Figura 12: Ieșirea `vo_trajectory.png` a scriptului: vederea de sus, cele trei componente ale poziției și eroarea în timp.

- **Keyframes înjumătățesc eroarea.** Așteptând suficient parallax, direcția translației devine o măsurătoare utilizabilă. Forma traiectoriei este reprodusă bine.
- **Mișcarea verticală rapidă se pierde.** În jurul secundei 30, drona coboară rapid în câteva imagini neclare. Puține match-uri supraviețuiesc, iar înălțimea estimată rămâne de atunci cu 0,9 m prea mare. Aceasta arată direct cât de mult depinde VO de calitatea keypoint detection și matching.
- **Heading drift.** La finalul zborului, yaw-ul estimat diferă cu aproximativ 33° de OptiTrack, iar fiecare pas ulterior este rotit cu această eroare.

9 Limitări și ce fac sistemele reale

Pipeline-ul este intenționat minimal. Sisteme reale precum ORB-SLAM, SVO sau VINS adaugă:

- **frame-to-map tracking**: punctele sunt triangulate într-o hartă 3D locală, iar imaginile noi sunt localizate față de hartă cu Perspective-n-Point (PnP), astfel încât scara rămâne consistentă;
- **bundle adjustment**: pose-urile și punctele 3D ale mai multor keyframes sunt optimizate împreună, ceea ce reduce mult drift-ul;
- **sensor fusion**: un IMU măsoară rotația în timpul mișcărilor rapide și face scara observabilă (visual-inertial odometry);
- **loop closure**: recunoașterea unui loc deja vizitat (de exemplu cu un bag of ORB words) permite eliminarea drift-ului acumulat; odometry devine SLAM.

10 Exerciții: ablation study

Un **ablation study** răspunde la întrebarea: *cât contează fiecare componentă?* Pornim de la pipeline-ul complet (baseline), apoi dezactivăm sau înlocuim **o singură componentă** și măsurăm cum se schimbă rezultatul. Exercițiile folosesc scriptul `Lab_3_VIO_obstacle_avoidance/6_visual_odometry_ablations.py`. Cu setările implicite, el face exact ce este descris în aceste note; în plus, are comutatoare pentru fiecare componentă:

Comutator	Valori	Ce schimbă
`--detector`	`orb` | `akaze` | `sift`	keypoints și descriptori (§3)
`--matcher`	`ratio` | `crosscheck`	cum se formează perechile (§4)
`--tracking`	`keyframe` | `previous`	comparăm cu keyframe-ul sau cu cadrul anterior (§7)
`--no-clahe`		fără creșterea contrastului
`--no-mask`		fără self mask: keypoints pot apărea pe elice
`--features`	1000	numărul maxim de keypoints pe imagine
`--ratio`	0,75	pragul ρ pentru ratio test
`--ransac-px`	1,0	pragul RANSAC [px]
`--min-parallax`, `--max-gap`	4,0; 8	regula pentru keyframes

Pregătire

În folderul `Lab_3_VIO_obstacle_avoidance`, dezarhivați `stream_out.zip`. Arhiva conține același zbor ca în aceste note (`stream_out/imgs/1794489705263`). Rulați apoi baseline-ul:

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video
```

Capcană

Arhiva din Lab 3 nu conține folderul `ranging/`. Fără el, scriptul folosește o calibrare implicită ($f = 180$ px, fără distorsiune) și **nu are self mask**, deci `--no-mask` nu schimbă nimic. Copiați folderul `ranging/` din `Lab_2_Follow_Me/stream_out/imgs/1794489705263/` în `stream_out/imgs/1794489705263/` înainte de a începe.

Reguli de lucru

- Schimbați **un singur lucru** la fiecare rulare și comparați întotdeauna cu baseline-ul.
- Folosiți `--no-video` pentru viteză. Rulați fără el doar când vreți să vedeți match-urile în `vo_matches.mp4`.
- Fiecare rulare adaugă un rând în `stream_out/imgs/1794489705263/vo_ablation_summary.csv`. Coloanele importante sunt `median_matches`, `median_inliers`, `median_err_rotated_m`, `final_err_rotated_m`, `final_yaw_err_deg` și `run_s`.

- Numele folderului de ieșire conține doar detectorul, matcher-ul, tracking-ul, CLAHE și masca. Când schimbați un parametru numeric (`--ratio`, `--features`, …), adăugați `--tag`, altfel rezultatul suprascrie folderul baseline-ului: `--ratio 0.6 --tag ratio06`.

Exerciții

1. **Baseline.** Rulați configurația implicită o dată *cu* video. Deschideți `vo_trajectory.png` și `vo_matches.mp4`. Notați numărul median de match-uri și de inliers și eroarea mediană. În ce momente ale zborului scade numărul de inliers și de ce?

2. **CLAHE** (§3).

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --no-clahe
```

   Cum se schimbă numărul de match-uri și de inliers? Dar eroarea? Explicați legătura cu FAST și pragul τ.

3. **Self mask.**

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-mask
```

   Urmăriți video-ul: apar match-uri pe elice sau pe ramă? Ce deplasare au ele între cele două imagini? Ce se întâmplă cu traiectoria estimată și de ce?

4. **Ratio test vs. cross-check** (§4).

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --ratio 0.6 --tag ratio06
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --ratio 0.9 --tag ratio09
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --matcher crosscheck
```

   Cu `crosscheck`, o pereche este păstrată dacă fiecare descriptor este cel mai bun match al celuilalt, fără ratio test. Calculați pentru fiecare rulare fracțiunea de inliers, `median_inliers / median_matches`. Ce metodă elimină mai bine match-urile ambigue? Cât de mult trebuie să „repare” RANSAC în fiecare caz?

5. **Numărul de keypoints.**

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --features 300 --tag f300
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --features 2000 --tag f2000
```

   Cum se schimbă numărul de inliers, timpul de rulare (`run_s`) și eroarea? Mai mulți keypoints înseamnă întotdeauna un rezultat mai bun?

6. **Detectorul.**

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --detector sift
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --detector akaze
```

   SIFT folosește descriptori float (Euclidean distance), AKAZE descriptori binari (Hamming distance). Comparați timpul pe pas și eroarea cu ORB. Merită viteza lui ORB pe o dronă? AKAZE necesită OpenCV 4 (`pip install "opencv-python<5"`); în OpenCV 5 scriptul afișează un mesaj și se oprește.

7. **Keyframes vs. cadru cu cadru** (§7).

```
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --tracking previous
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --min-parallax 2 --tag par2
python 6_visual_odometry_ablations.py stream_out/imgs/1794489705263 --no-video --min-parallax 8 --tag par8
```

   Cu `previous`, fiecare imagine este comparată cu cea anterioară. Cum se schimbă numărul de pași, numărul de inliers și eroarea? Explicați rezultatul cu ajutorul Figurii 11.

8. **Sinteză.** Completați tabelul de mai jos din `vo_ablation_summary.csv`. Ordonați componentele după cât de mult crește eroarea când sunt eliminate. Care componentă contează cel mai mult pentru acest zbor? Care aproape deloc? Scrieți o concluzie de 5–10 rânduri.

Configurație	match-uri	inliers	eroare mediană [m]	eroare finală [m]	eroare yaw [°]	ms/pas
baseline (ORB, ratio, keyframe)						
`--no-clahe`						
`--no-mask`						
`--ratio 0.6` / `0.9`						
`--matcher crosscheck`						
`--features 300` / `2000`						
`--detector sift`						
`--detector akaze`						
`--tracking previous`						
`--min-parallax 2` / `8`						

A Rularea codului

```
cd Lab_2_Follow_Me
.venv/bin/python 12_visual_odometry_orb.py stream_out/imgs/1794489705263
.venv/bin/python 12_visual_odometry_orb.py stream_out/imgs/1794489705263 \
    --min-parallax 2 --max-gap 3 --features 1500
```

Rezultatele sunt scrise în `<session>/vo/`: `vo_trajectory.csv` (un rând pentru fiecare pas acceptat), `vo_trajectory.png` și `vo_matches.mp4` (keyframe-ul și imaginea curentă una lângă alta, cu inliers în verde).

opțiune	implicit	semnificație
`--features`	1000	numărul maxim de ORB keypoints pe imagine
`--ratio`	0,75	pragul ρ pentru Lowe's ratio test
`--ransac-px`	1,0	pragul RANSAC pentru inliers ϵ [px]
`--min-inliers`	20	sub acest număr de inliers, pasul este sărit
`--min-parallax`	4,0	parallax-ul [px] necesar pentru un keyframe nou
`--max-gap`	8	numărul maxim de cadre între două keyframes

Tabela 2: Opțiunile din linia de comandă.

secțiunea din aceste note	funcția din `12_visual_odometry_orb.py`
keypoint detection (§3)	`load` în `main`
matching (§4)	`match`
essential matrix, pose (§5, §6)	`relative_pose`
scară, keyframes, integrare (§7)	`optitrack_at`, bucla principală

Tabela 3: Unde se găsește fiecare pas în cod.

Bibliografie

[1] E. Rublee, V. Rabaud, K. Konolige, G. Bradski. ORB: an efficient alternative to SIFT or SURF. *ICCV*, 2011.

[2] E. Rosten, T. Drummond. Machine learning for high-speed corner detection. *ECCV*, 2006.

[3] M. Calonder, V. Lepetit, C. Strecha, P. Fua. BRIEF: binary robust independent elementary features. *ECCV*, 2010.

[4] C. Harris, M. Stephens. A combined corner and edge detector. *Alvey Vision Conference*, 1988.
[5] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *IJCV*, 60(2), 2004.
[6] D. Nistér, O. Naroditsky, J. Bergen. Visual odometry. *CVPR*, 2004.
[7] D. Nistér. An efficient solution to the five-point relative pose problem. *IEEE TPAMI*, 26(6), 2004.
[8] M. A. Fischler, R. C. Bolles. Random sample consensus. *Communications of the ACM*, 24(6), 1981.
[9] H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293, 1981.
[10] D. Scaramuzza, F. Fraundorfer. Visual odometry: Part I — the first 30 years and fundamentals. *IEEE Robotics & Automation Magazine*, 18(4), 2011.
[11] R. Mur-Artal, J. M. M. Montiel, J. D. Tardós. ORB-SLAM: a versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics*, 31(5), 2015.